

FOR SOFTWARE ENGINEERS: THE HALF ANSWER MOST SENIORS GIVE

# The Quietly Quadratic Loop

It compiles, it is correct, and it passes review

Double the input and the time roughly triples. That is a complexity class changing, in wall clock rather than in a textbook.

Most people look at this loop and assume the problem is `String res = ""`. It is not. The problem is `res +=` inside the loop.

Saying "this is slow" is not the answer. Saying WHY the complexity class changes is, and that is a language rule almost every engineer can recite and very few apply.

The timings on this page were measured on one machine in one session, on Java 21. They are not illustrative.

## In scope

- Why immutability turns a linear-looking loop into quadratic work
- Three measurements showing the growth in wall clock
- Why sizing the builder up front still matters
- The one-line answer that exists for this exact case since Java 11
- Why the compiler optimisation people cite does not rescue this

## Out of scope

- Micro-benchmark methodology. These are wall clock timings, stated as such
- Any claim about your hardware. The SHAPE of the growth is the transferable part
- String interning, which is a different question that sounds related
- Rope or builder implementations in other languages

Read the table first. The absolute milliseconds belong to one machine; the ratio between the rows belongs to everybody.



## 01 Measured on Java 21, one machine, one session

|                           |                                           |
|---------------------------|-------------------------------------------|
| n = 10,000, res += s      | 29.3 ms                                   |
| n = 20,000, res += s      | 46.5 ms                                   |
| n = 40,000, res += s      | 135.8 ms                                  |
| n = 40,000, StringBuilder | 0.833 ms                                  |
| The ratio that matters    | About 163 times faster at n = 40,000      |
| The shape that matters    | Double the input, roughly triple the time |

Each doubling of n roughly triples the time for the concatenation, which is the signature of quadratic growth. The builder does not move.

### 03 Why it is quadratic

Strings in Java are immutable. Every `+=` creates a NEW String and copies everything accumulated so far into it.

On iteration one you copy one unit. On iteration two, two. The total work is the sum of one to  $n$ , which is quadratic in the LENGTH OF THE OUTPUT rather than linear in the number of iterations.

That distinction is the whole answer. The loop runs  $n$  times and looks linear. The work inside it grows with what has already been built, and that is what nobody says out loud.

String objects are immutable, so their contents cannot be changed after they are created.

`The Java Language Specification, String objects are immutable`

Summing one to  $n$  is quadratic in  $n$ , so copying an accumulating buffer once per iteration is quadratic in the length of the result.

`First principles, not vendor documentation`

### 04 Measured, not asserted

Ten thousand took 29.3 milliseconds. Twenty thousand took 46.5. Forty thousand took 135.8.

Double the input and the time roughly triples. That is quadratic growth appearing in wall clock rather than in a complexity table, and it is far more persuasive in a room than the theory is.

The builder at the same size took 0.833 milliseconds, and stays flat as the input grows.

At  $n = 40,000$  the concatenating loop took 135.8 ms and the builder took 0.833 ms on the same machine in the same session.

`Measured on the machine that built this sheet, Java 21, not quoted`

The absolute numbers belong to one machine. The ratio between the rows is the part that transfers.

`First principles, not vendor documentation`

## 05 The fix, and the detail inside the fix

Use a `StringBuilder` and `append`. The buffer is mutable, so nothing is copied on each iteration.

SIZE IT UP FRONT. ``new StringBuilder(s.length() * n)`` matters, because without a capacity the internal array still has to grow and copy. Far fewer times than the concatenation did, but not zero.

And for this exact case, since Java 11, the shorter answer is ``s.repeat(n)``. Knowing that exists is worth a sentence, because reaching for a builder when the library already has the method is its own small tell.

`StringBuilder` provides a mutable sequence of characters, so appending does not copy the accumulated content on every call.

`StringBuilder`, [API documentation](#)

A builder created without a capacity still resizes its internal array as it grows, which is why sizing it up front is not a micro-optimisation.

`StringBuilder`, [API documentation](#)

`String.repeat` returns a string whose value is the concatenation of this string repeated `n` times, and has existed since Java 11.

`String.repeat`, [introduced in Java 11](#)

## 06 Why the optimisation people cite does not rescue you

Someone will say the compiler turns string concatenation into a builder anyway. That is true WITHIN one expression.

Here the concatenation happens ACROSS LOOP ITERATIONS. Each iteration is its own expression, so each one builds, copies and discards. There is nothing for that optimisation to fuse.

Being able to say that sentence is the difference between having heard about the optimisation and understanding what it applies to.

The compiler may optimise a concatenation expression into a builder, which does not extend across separate statements executed in separate iterations.

[The Java Language Specification](#), [string concatenation operator](#)



## 07 What the interviewer is listening for

Not "use a StringBuilder". Almost everyone gets there, and getting there proves very little.

The answer that separates people names three things: immutability, the copy on every iteration, and the resulting change of complexity class. Then it adds that the compiler optimisation does not apply because the concatenation is across iterations rather than within one expression.

If you have a number, use it. "It roughly triples each time the input doubles" is a sentence that sounds like someone who ran it.

Naming the mechanism and the complexity class, rather than the remedy, is what distinguishes a measured answer from a memorised one.

First principles, not vendor documentation

### Before you call a loop linear

1. Does anything inside the loop copy what has already been accumulated?
2. Is the accumulating type immutable?
3. Is the complexity in the number of iterations, or in the size of the result?
4. Would doubling the input double the time, or more than double it?
5. Is there a capacity I could give the builder up front?
6. Does the standard library already have this method?
7. Am I relying on a compiler optimisation that works only within one expression?
8. Have I measured it, or am I reasoning about it?
9. Can I state the complexity class, not just that it is slow?
10. Does the same argument apply to the list or map I am building the same way?

## Everything else

### Site

<https://code-with-sam-dev.github.io>

### YouTube

<https://www.youtube.com/@CodewithSam-Dev>

### LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

### X

<https://x.com/CodeWithSamDev>

### TikTok

<https://www.tiktok.com/@codewithsamdev>

### Instagram

<https://www.instagram.com/codewithsamdev>

### Facebook

<https://www.facebook.com/codewithsamdev>

### GitHub

<https://github.com/code-with-sam-dev>



### Buy me a coffee

<https://buymeacoffee.com/codewithsam>

### Buy me sadza

<https://buymeacoffee.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of these you have actually shipped.

Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates. Eclipse Temurin is a trademark of the Eclipse Foundation. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-16 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.